ЗВІТ

Практична робота №15

1. Виконання роботи

Створено папку lab34 з такими файлами:

- expenses.py — функції для обробки витрат;
- test_expenses.py — автоматичні тести;
- data/expenses.csv — файл із власними витратами;
- requirements.txt — список залежностей проєкту.

У файлі expenses.py реалізовано три функції:

- parse_line() — розбирає рядок із даними про витрати;
- total_by_category() — обчислює загальну суму витрат за категоріями;
- load_expenses() — зчитує витрати з CSV-файлу.

2. Тестування програми

Для перевірки використано бібліотеку pytest.

Створено тести для:

- правильного розбору рядка;
- перевірки неправильної кількості полів;
- перевірки некоректної суми;
- параметризації п'яти наборів даних;
- підрахунку витрат за категоріями;
- читання тимчасового файлу за допомогою tmp_path;
- перевірки власного файлу expenses.csv.

Команда запуску тестів:

pytest -v

Результат успішного тестування:

11 passed in 0.02s

Усі 11 тестів пройшли успішно.

3. Перевірка виявлення помилок

Для демонстрації роботи тестів було тимчасово змінено тип суми: замість числа float функція повертала текст str.

У результаті тестування отримано:

8 failed, 3 passed in 0.09s

Виникли помилки AssertionError та TypeError, оскільки тести виявили неправильний тип даних.

Після цього початковий код було відновлено. Повторний запуск показав 11 passed.

4. Перевірка власних витрат

Для тестування використано власний файл data/expenses.csv.

Отримані результати:

Категорія	Сума
food	590
transport	96
coffee	60
entertainment	230
mobile	100
Разом	**1076**

Кількість записів — 12.

5. Відповіді на контрольні запитання

1. Чим модульний тест відрізняється від ручної перевірки?

Модульний тест автоматично перевіряє окрему частину програми, а ручну перевірку людина виконує самостійно.

2. Як pytest знаходить тести?

За замовчуванням pytest шукає файли test_*.py або *_test.py, а також функції, назви яких починаються з test_.

3. Чому тест має бути здатний спочатку впасти?

Це доводить, що тест справді може виявити помилку в програмі.

4. Як перевірити, що функція викликає виняток?

За допомогою pytest.raises(), наприклад:

with pytest.raises(ValueError):

5. Навіщо потрібна параметризація?

Вона дозволяє перевірити одну функцію на різних наборах даних без дублювання коду.

6. Чому для тестів використовують tmp_path?

tmp_path створює тимчасову папку для тестових файлів, тому реальні файли користувача не змінюються.